Homework 3 Solution

This notebook gives instructor solutions for PCA/SVD dimension reduction on a synthetic data set (Problem 1) and on fluid flow past a cylinder (Problem 2).

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat

np.random.seed(0)
HW = '/Users/xxie/Documents/teaching/BDA411:511/homework/'   # location of the original
data
plt.rcParams.update({'font.size': 11})
```

Problem 1: PCA on the 100 by 2000 data set

Setup

The data matrix $X \in \mathbb{R}^{100 \times 2000}$ has 100 samples (rows) and 2000 features (columns). PCA centers each feature, $X_c = X - \mathbf{1}\bar{x}^\top$, and takes the SVD $X_c = U\Sigma V^\top$. The rank-r approximation is

$$\tilde{X}_r = U_r \Sigma_r V_r^\top + \mathbf{1}\bar{x}^\top.$$

Because there are only 100 samples, $\operatorname{rank}(X_c) \le 99$. Therefore, for every $r \ge 99$ the reconstruction is exact. The prescribed list `r = np.arange(1,1000,5)` goes beyond this, so we cap the number of components at $\min(r, 100)$.

```
X = np.load(HW + 'hwk3data.npy')
print('shape:', X.shape)
mu = X.mean(axis=0)
Xc = X - mu
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
print('number of singular values:', len(s), ' numerical rank:', np.sum(s > 1e-10*s[0]))

rs = np.arange(1, 1000, 5)
err_fro = []
for r in rs:
    k = min(r, len(s))
    Xr = (U[:, :k] * s[:k]) @ Vt[:k] + mu
    err_fro.append(np.linalg.norm(X - Xr, 'fro') / np.linalg.norm(X, 'fro'))
err_fro = np.array(err_fro)
# cross-check with singular values: ||Xc - Xc_r||_F^2 = sum of discarded sigma^2
tail = np.array([np.sum(s[min(r,len(s)):]**2) for r in rs])
print('max |direct - from sigma| (squared absolute error):',
      np.max(np.abs((err_fro*np.linalg.norm(X,'fro'))**2 - tail)))
for r in [1, 6, 11, 51, 96, 101]:
    print(f'r={r:4d}  relative error (vs ||X||_F) = {err_fro[list(rs).index(r)]:.4e}')
```

```
shape: (100, 2000)
```

```
number of singular values: 100  numerical rank: 99
```

```
max |direct − from sigma| (squared absolute error): 1.1026859283447266e-06
r=   1  relative error (vs ||X||_F) = 9.8597e-01
r=   6  relative error (vs ||X||_F) = 9.3935e-01
r=  11  relative error (vs ||X||_F) = 8.9524e-01
r=  51  relative error (vs ||X||_F) = 5.6872e-01
r=  96  relative error (vs ||X||_F) = 1.1427e-01
r= 101  relative error (vs ||X||_F) = 3.3854e-15
```

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(rs, err_fro, 'o-', ms=3)
ax[0].set_xlabel('truncation r'); ax[0].set_ylabel('relative Frobenius error')
ax[0].set_title('Reconstruction error versus r'); ax[0].grid(alpha=.3)
ax[1].semilogy(rs, err_fro + 1e-17, 's--', ms=3, color='k')
ax[1].axvline(99, ls=':', color='gray', label='rank of centered data = 99')
ax[1].set_xlabel('truncation r'); ax[1].set_ylabel('relative Frobenius error (log scale)')
ax[1].set_title('Same curve, logarithmic vertical axis'); ax[1].legend();
ax[1].grid(alpha=.3)
plt.tight_layout(); plt.show()
```

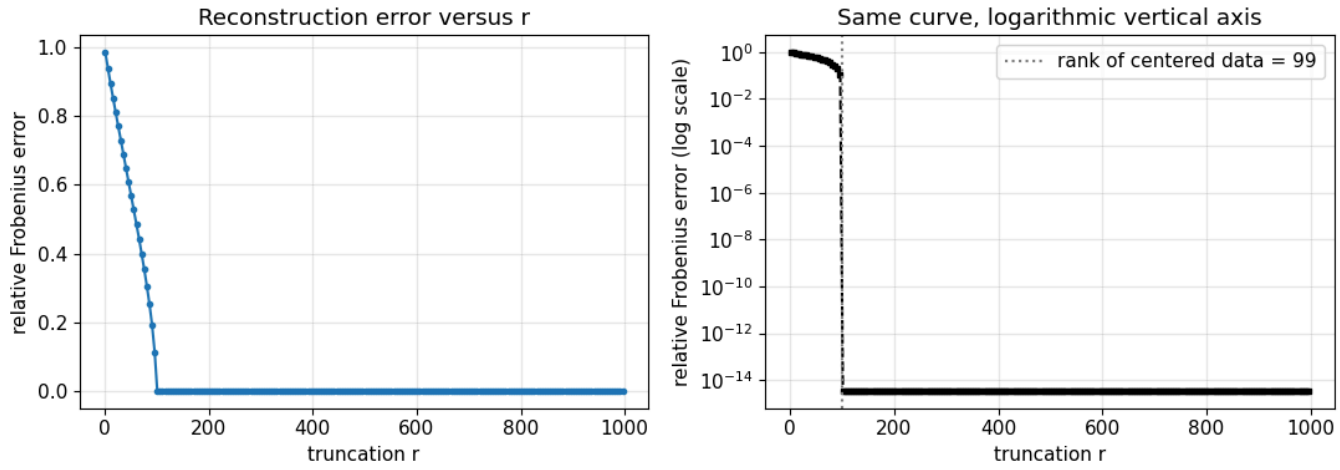


Figure description. Two line plots of relative reconstruction error against the truncation value r. The left plot (linear axis, circle markers) shows the error decreasing steadily from a large value at small r to essentially zero near $r = 100$, then staying at zero. The right plot (log axis, dashed line with square markers) shows the same curve and a dotted vertical line at $r = 99$, the rank of the centered data, after which the error is at machine precision.

Discussion

The error decreases monotonically with r (Eckart-Young: the truncated SVD is the best rank-r approximation) and drops to zero once r reaches the rank, 99. Values $r > 100$ in the prescribed grid add nothing, because the data have only 100 samples. A slowly decaying spectrum (next figure) means the data are not low-rank, so many components are needed for an accurate reconstruction.

```
plt.figure(figsize=(6,4))
plt.semilogy(np.arange(1,len(s)+1), s, 'o-', ms=3)
plt.xlabel('index i'); plt.ylabel(r'singular value $\sigma_i$')
plt.title('Singular values of the centered data'); plt.grid(alpha=.3); plt.show()
```

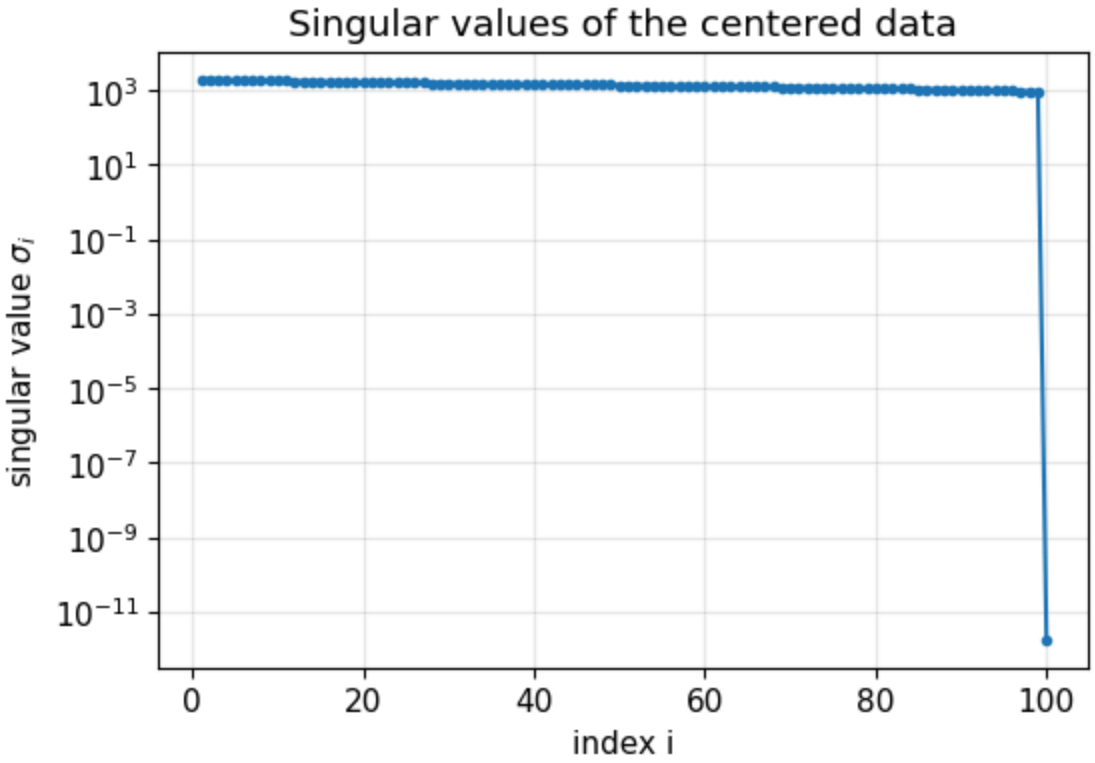


Figure description. A semilog line plot with circle markers of the singular values of the centered data matrix, with the last one at numerical zero. The values decay gradually with no sharp gap, confirming that no small number of components captures most of the data.

Problem 2: SVD of flow past a cylinder

Load and inspect the data

The file contains UALL, VALL (velocity components, 89351 by 151) and VORTALL (vorticity), each column being one snapshot of a 449×199 grid, plus a single extra snapshot for each (UEXTRA, VEXTRA, VORTEXTRA) that is not part of the 151 columns.

Note on the problem statement: the code snippet assigns `uextra = data2D['VEXTRA']` and `vextra = data2D['UEXTRA']`. The names are swapped. We use the correct pairing. The snippet's axis labels ('x' and 't') are also misleading, since the vertical axis is the spatial coordinate y.

We take the data matrix to be $X =$ UALL, i.e. the u-velocity movie with $m = 89351$ grid points and 151 snapshots.

```
d = loadmat(HW + 'CYLINDER_ALL.mat')
X = d['UALL'];  nx, ny = d['nx'].item(), d['ny'].item()
print('X shape:', X.shape, ' grid:', nx, 'x', ny, ' nx*ny =', nx*ny)
u = X.reshape(449, 199, -1)
uextra = d['UEXTRA'].reshape(449, 199, -1)   # corrected name (snippet swaps U/V)
print('u cube:', u.shape, ' extra snapshot:', uextra.shape)

def show(ax, field, title, vmin=None, vmax=None):
    im = ax.pcolormesh(field.T, cmap='jet', vmin=vmin, vmax=vmax)
    ax.set_title(title); ax.set_xlabel('x (grid index)'); ax.set_ylabel('y (grid index)')
    return im
fig, ax = plt.subplots(figsize=(8,4))
im = show(ax, u[:,:,0], 'Velocity component u, first snapshot'); plt.colorbar(im, ax=ax,
label='u')
plt.tight_layout(); plt.show()
```

```
X shape: (89351, 151)  grid: 199 x 449  nx*ny = 89351
u cube: (449, 199, 151)  extra snapshot: (449, 199, 1)
```

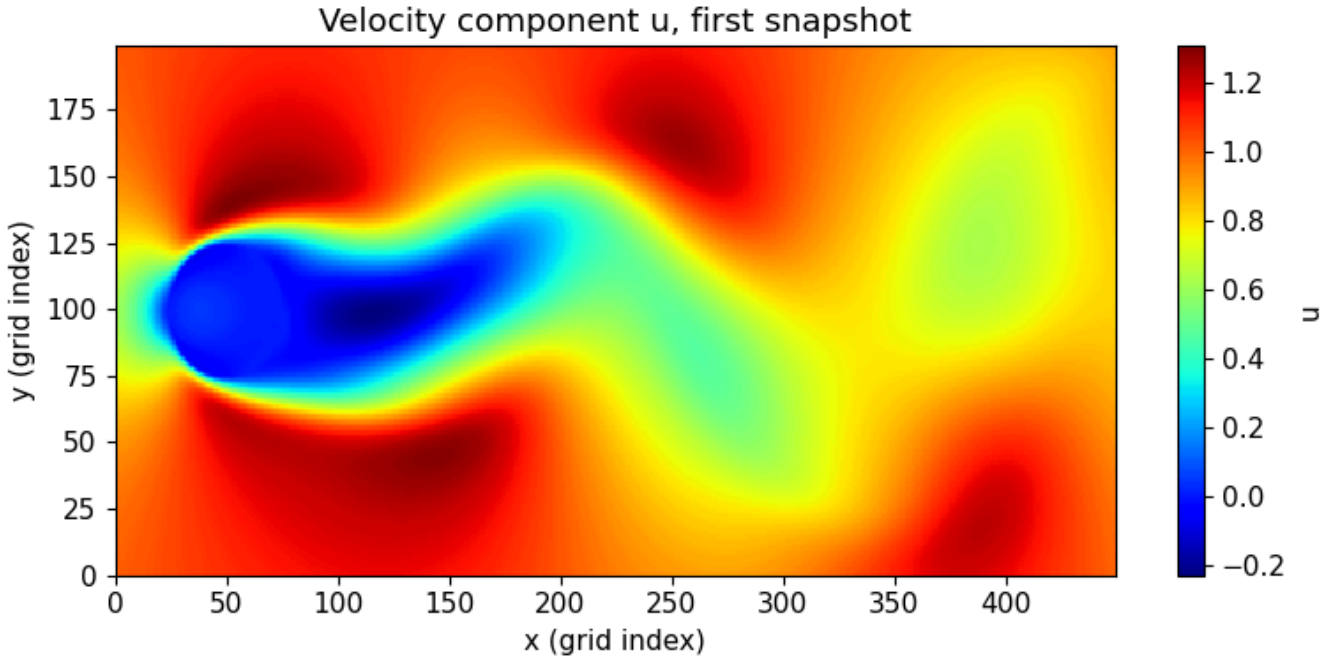


Figure description. A color map (jet colormap) of the horizontal velocity u in the first snapshot over the 449×199 grid. A wake of alternating high and low velocity regions trails behind the cylinder, which sits near the left of the domain; the colorbar gives the value of u.

(a) SVD, singular value spectrum and leading modes

We compute the economy SVD $X = U\Sigma V^*$ without subtracting the mean (as in the problem statement), so the first mode captures the mean flow.

```
U, s, Vt = np.linalg.svd(X, full_matrices=False)
print(U.shape, s.shape, Vt.shape)
print('first 6 singular values:', np.round(s[:6], 2))
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].semilogy(np.arange(1,len(s)+1), s, 'o-', ms=3)
ax[0].set_xlabel('index i'); ax[0].set_ylabel(r'$\sigma_i$'); ax[0].set_title('Singular
value spectrum (log scale)'); ax[0].grid(alpha=.3)
ax[1].plot(np.arange(1,len(s)+1), np.cumsum(s**2)/np.sum(s**2), 's--', ms=3, color='k')
ax[1].set_xlabel('r'); ax[1].set_ylabel('cumulative energy fraction');
ax[1].set_title('Cumulative energy'); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.show()
```

```
(89351, 151) (151,) (151, 151)
first 6 singular values: [3265.48  339.71  334.69   84.57   83.6    33.48]
```

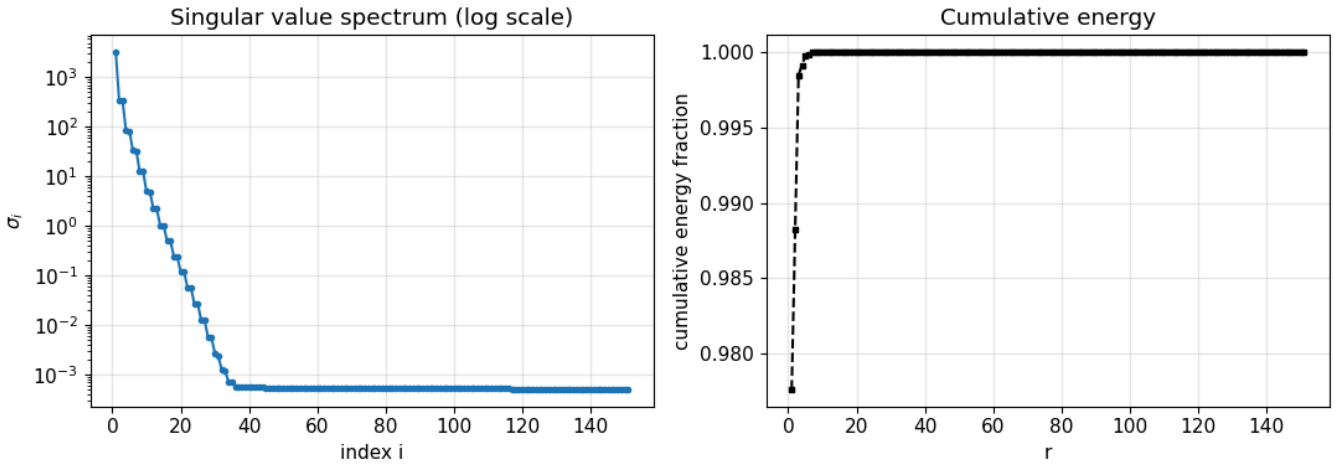


Figure description. Left: a semilog plot (circle markers) of the 151 singular values, with a very large first value (about 3265, the mean flow), then values near 340, 335, 85, 84 and 33 that come in pairs of nearly equal size (shedding modes), followed by a rapid decay to very small values. Right: the cumulative energy fraction (dashed line, square markers) rising to about 0.98 with the first mode alone and to nearly 1 within the first few modes.

```
fig, axs = plt.subplots(3, 2, figsize=(11, 9))
for i, ax in enumerate(axs.ravel()):
    mode = U[:, i].reshape(449, 199)
    v = np.max(np.abs(mode))
    im = show(ax, mode, f'Left singular vector (mode) {i+1}', -v, v)
    plt.colorbar(im, ax=ax)
plt.tight_layout(); plt.show()
```

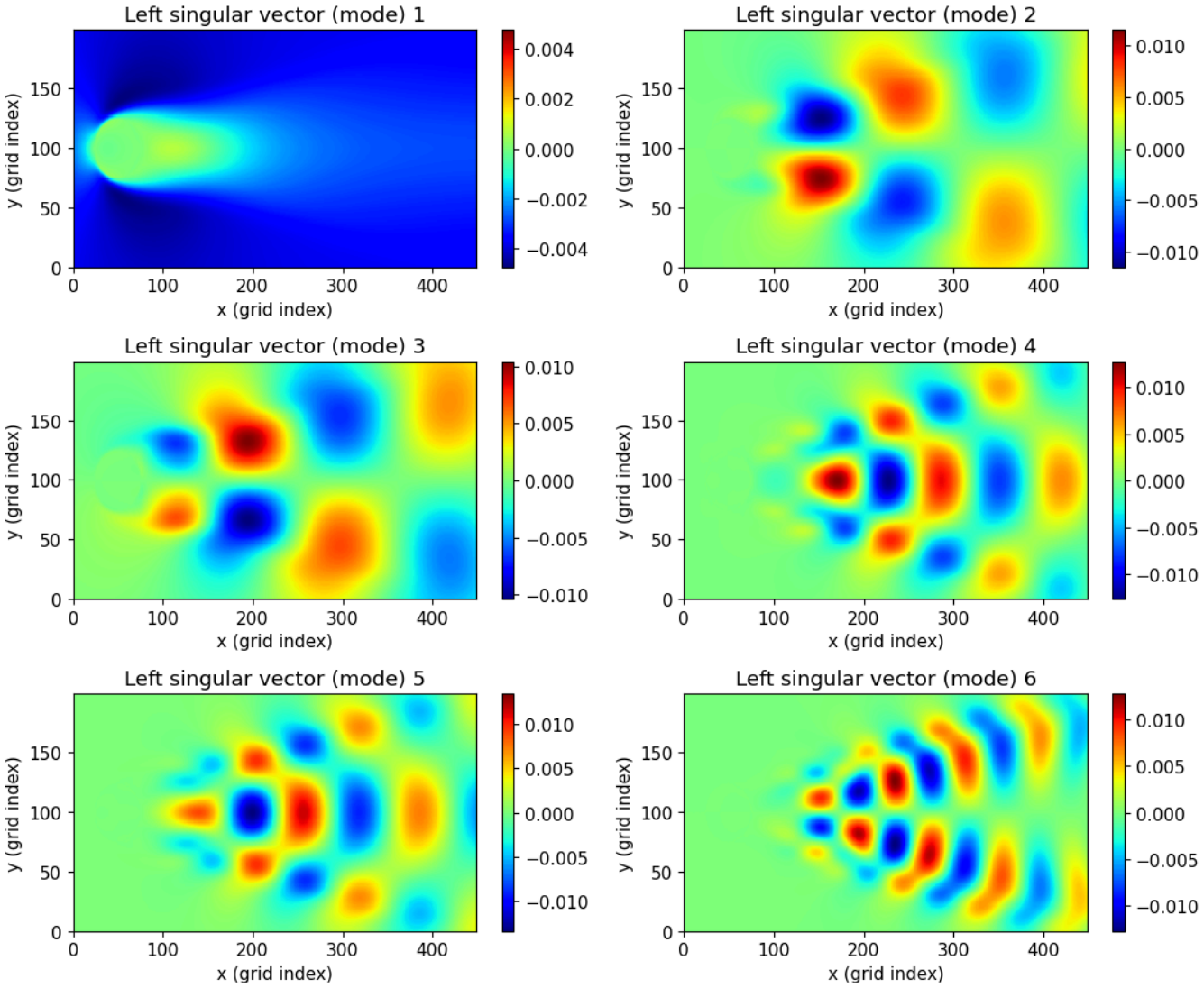


Figure description. Six color maps of the first six left singular vectors (spatial modes) of the velocity field. Mode 1 is the mean flow. Later modes come in pairs of similar structures shifted along x, representing the travelling vortex shedding in the wake. Higher modes show progressively finer spatial structure.

(b) Reconstructions, energy thresholds and error

The rank-r reconstruction is $\tilde{X}_r = U_r \Sigma_r V_r^*$. By the Eckart-Young theorem the squared Frobenius error is $\| X - \tilde{X}_r \|_F^2 = \sum_{i>r} \sigma_i^2$, and the energy is $\| X \|_F^2 = \sum_i \sigma_i^2$. The smallest r capturing a fraction p of the energy satisfies $\sum_{i \le r} \sigma_i^2 \ge p \, \| X \|_F^2$.

```
energy = np.cumsum(s**2) / np.sum(s**2)
for p in [0.90, 0.99, 0.999]:
    r_p = int(np.searchsorted(energy, p) + 1)
    Xr = (U[:, :r_p] * s[:r_p]) @ Vt[:r_p]
    err_direct = np.linalg.norm(X - Xr, 'fro')**2
    err_sigma = np.sum(s[r_p:]**2)
    print(f'{p*100:5.1f}% energy: r = {r_p:3d},  ||X - Xr||_F^2 = {err_direct:.4e} (from
sigma: {err_sigma:.4e}),'
          f' relative = {err_direct/np.linalg.norm(X,"fro")**2:.2e}')
print('total energy ||X||_F^2 =', f'{np.linalg.norm(X,"fro")**2:.4e}')
```

```
 90.0% energy: r =   1,  ||X - Xr||_F^2 = 2.4417e+05 (from sigma: 2.4417e+05), relative =
2.24e-02
```

```
 99.0% energy: r =   3,  ||X - Xr||_F^2 = 1.6753e+04 (from sigma: 1.6753e+04), relative =
1.54e-03
```

```
 99.9% energy: r =   4,  ||X - Xr||_F^2 = 9.6012e+03 (from sigma: 9.6012e+03), relative =
8.80e-04
total energy ||X||_F^2 = 1.0908e+07
```

```
rs_show = [1, 2, 5, 10, 20, 50]
k = 75   # snapshot (frame) shown
fig, axs = plt.subplots(4, 2, figsize=(11, 12))
axs = axs.ravel()
vmin, vmax = X[:, k].min(), X[:, k].max()
im = show(axs[0], X[:, k].reshape(449,199), f'Original snapshot k={k}', vmin, vmax)
for ax, r in zip(axs[1:], rs_show + [151]):
    Xr_k = (U[:, :r] * s[:r]) @ Vt[:r, k]
    show(ax, Xr_k.reshape(449,199), f'Rank r={r} reconstruction, snapshot k={k}', vmin,
vmax)
fig.colorbar(im, ax=axs, label='u', shrink=.6)
plt.show()
```

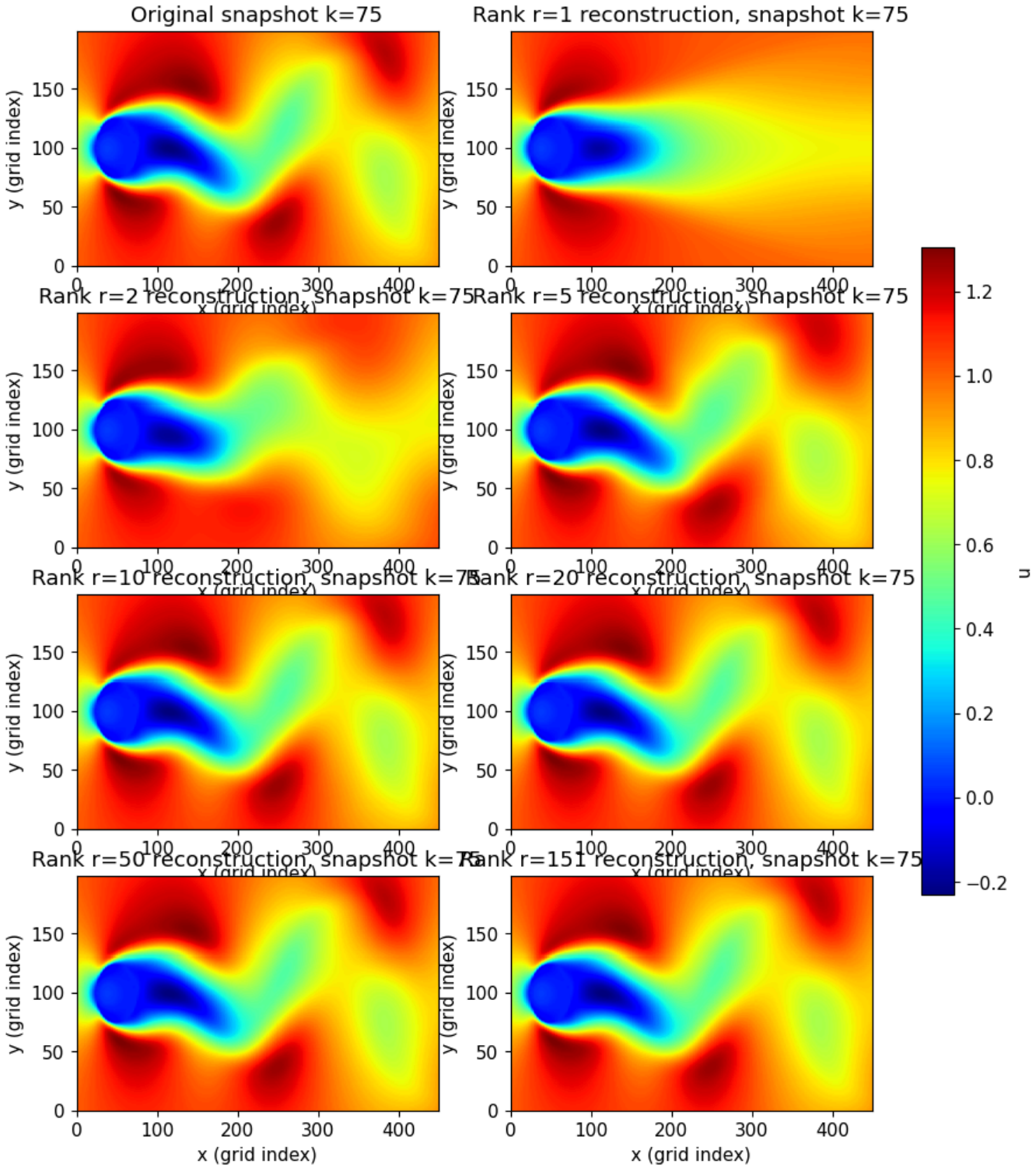


Figure description. A grid of eight color maps of snapshot 75 of the velocity field: the original, and reconstructions with $r = 1, 2, 5, 10, 20, 50$ and 151 (full rank). With $r = 1$ only the smooth mean flow is seen; as r grows the alternating vortex pattern in the wake appears, by $r = 10$ the picture is visually indistinguishable from the original, and $r = 151$ is identical to it. All panels share one color scale. This static grid stands in for a reconstructed movie, which would apply the same formula to every column.

```
# "Movie" as a function: reconstruct all frames for a given r (frames could be passed to
matplotlib.animation).
def reconstruct_movie(r):
    return ((U[:, :r] * s[:r]) @ Vt[:r]).reshape(449, 199, -1)
for r in [1, 5, 10, 20, 50, 100, 151]:
    err = np.sum(s[r:]**2)
    print(f'r={r:3d}  squared Frobenius error = {err:.4e}   relative =
{err/np.sum(s**2):.3e}')
print('check movie shape:', reconstruct_movie(10).shape)
```

```
r=  1  squared Frobenius error = 2.4417e+05   relative = 2.239e-02
r=  5  squared Frobenius error = 2.6121e+03   relative = 2.395e-04
r= 10  squared Frobenius error = 3.7405e+01   relative = 3.429e-06
r= 20  squared Frobenius error = 2.2740e-02   relative = 2.085e-09
r= 50  squared Frobenius error = 2.7915e-05   relative = 2.559e-12
r=100  squared Frobenius error = 1.3565e-05   relative = 1.244e-12
r=151  squared Frobenius error = 0.0000e+00   relative = 0.000e+00
check movie shape: (449, 199, 151)
```

(c) Rank-10 truncation and the coefficient matrix W

With $r = 10$ let $\tilde{U} = U_{:,1:10}$, $\tilde{\Sigma} = \mathrm{diag}(\sigma_1, ..., \sigma_{10})$ and $\tilde{V} = V_{:,1:10}$. The matrix $W = \tilde{\Sigma}\tilde{V}^* \in \mathbb{R}^{10\times 151}$ has columns w_k giving the mixture of the 10 eigenflows (columns of $\tilde{U}$) in snapshot k, so $x_k \approx \tilde{U} w_k$.

```
r = 10
Ut, St, Vtt = U[:, :r], np.diag(s[:r]), Vt[:r]
W = St @ Vtt
print('W shape:', W.shape)
Xtilde = Ut @ W
print('||X - Xtilde||_F^2 =', f'{np.linalg.norm(X-Xtilde,"fro")**2:.4e}', ' sum of
discarded sigma^2 =', f'{np.sum(s[r:]**2):.4e}')
ks = [0, 50, 100, 150]
fig, axs = plt.subplots(len(ks), 3, figsize=(13, 11))
for row, k in zip(axs, ks):
    a, b = X[:, k], Xtilde[:, k]
    vmin, vmax = a.min(), a.max()
    show(row[0], a.reshape(449,199), f'Original, k={k}', vmin, vmax)
    show(row[1], b.reshape(449,199), f'Rank-10 reconstruction, k={k}', vmin, vmax)
    e = (a-b).reshape(449,199); m_ = np.abs(e).max()
    im = show(row[2], e, f'Error, k={k} (rel. {np.linalg.norm(a-
b)/np.linalg.norm(a):.1%})', -m_, m_)
    plt.colorbar(im, ax=row[2])
plt.tight_layout(); plt.show()
```

```
W shape: (10, 151)
```

```
||X - Xtilde||_F^2 = 3.7405e+01  sum of discarded sigma^2 = 3.7405e+01
```

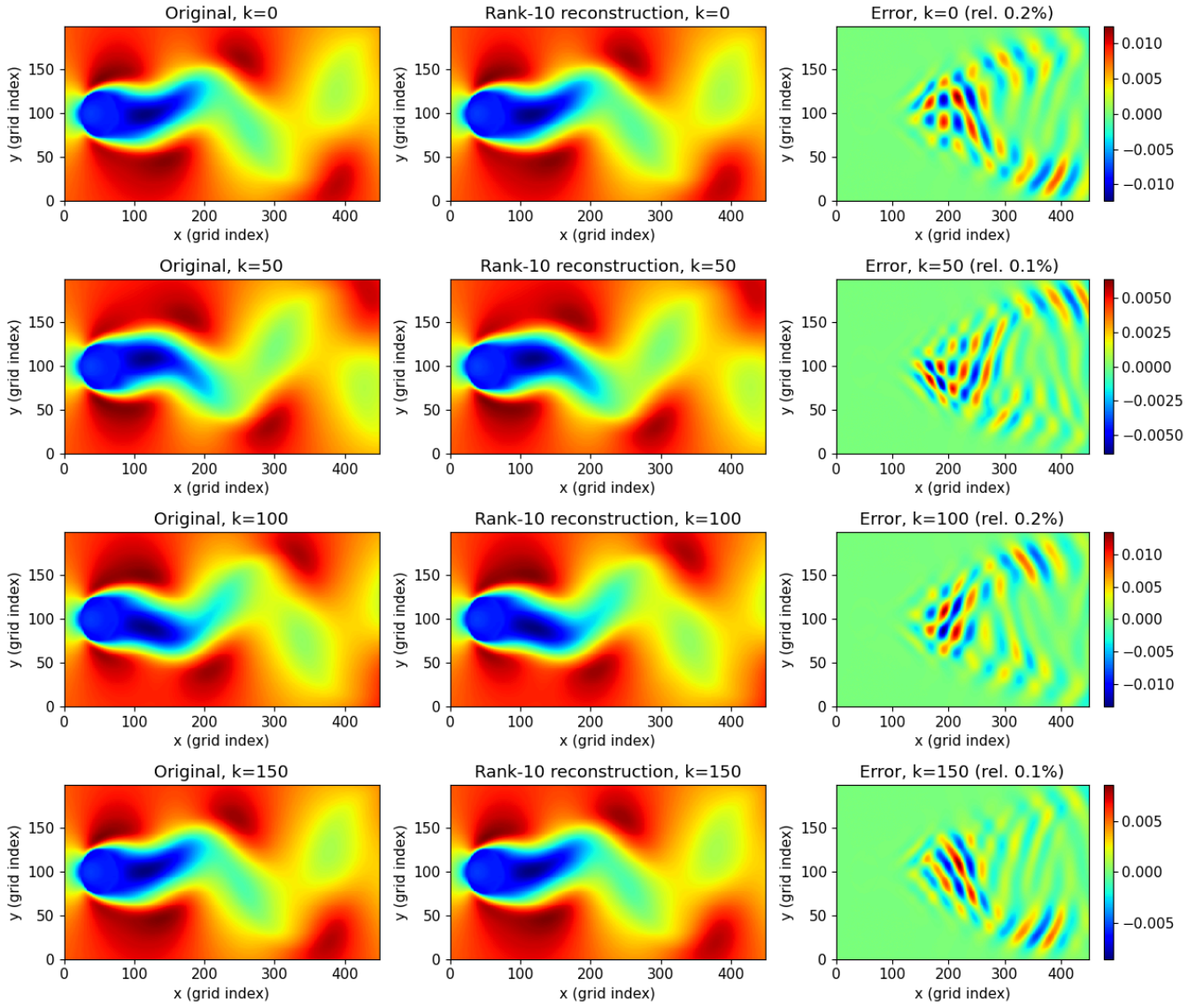


Figure description. A 4 by 3 grid of color maps for snapshots $k = 0, 50, 100, 150$. Columns show the original snapshot, its rank-10 reconstruction, and the pointwise error. The original and rank-10 panels look nearly identical; the error panels use a symmetric color scale and show very small residuals (relative error of order 10^{-3} or less, printed in each title).

```
fig, ax = plt.subplots(figsize=(8,4))
for i, mk in zip(range(4), ['o','s','^','d']):
    ax.plot(W[i], marker=mk, ms=3, lw=1, label=f'component {i+1}')
ax.set_xlabel('snapshot index k'); ax.set_ylabel(r'$w_k$ entry'); ax.set_title('First four
rows of W across snapshots')
ax.legend(); ax.grid(alpha=.3); plt.show()
```

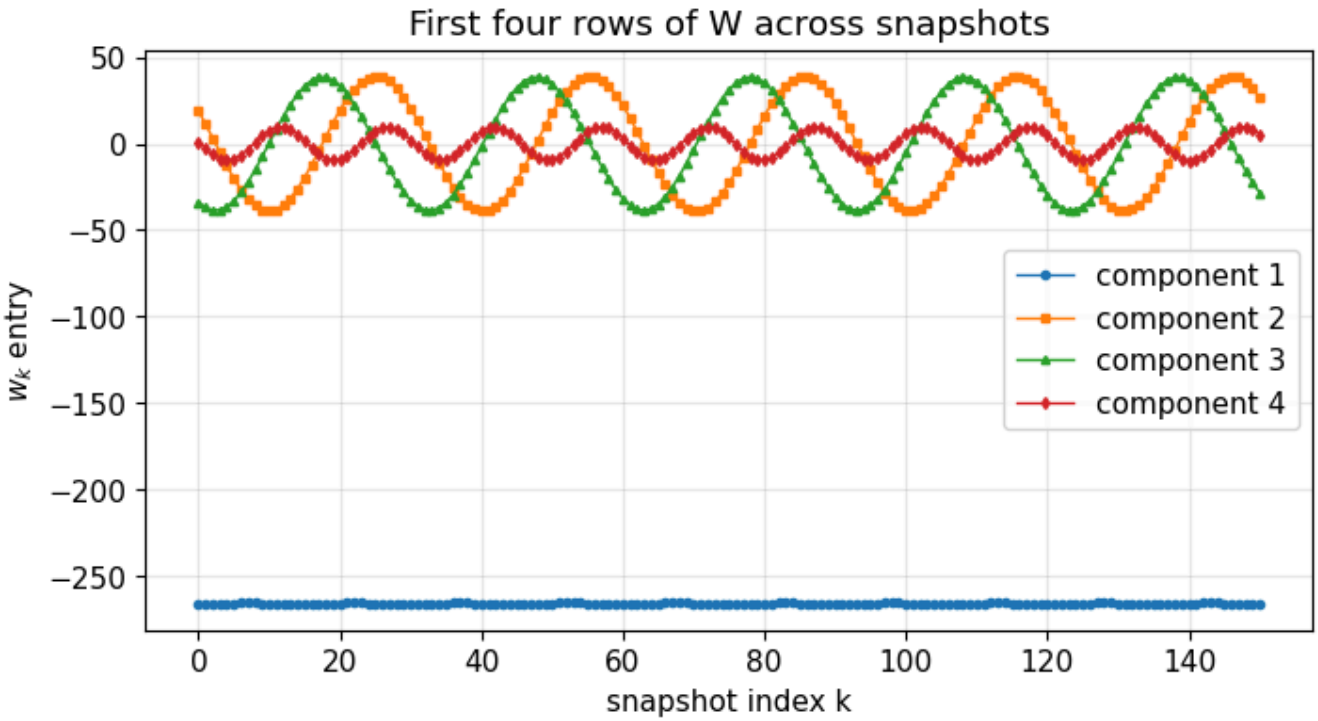


Figure description. Line plot with distinct markers of the first four rows of W against snapshot index. Component 1 is nearly constant (the mean flow), while components 2 to 4 oscillate with time, which reflects periodic vortex shedding.

Summary

- The data are well approximated by a low-rank model: the first mode is the mean flow and the next modes come in pairs describing periodic shedding.
- The ranks for 90%, 99% and 99.9% energy are $r = 1$, 3 and 4 (squared Frobenius errors 2.44×10^5, 1.68×10^4 and 9.60×10^3 out of $\| X \|_F^2 = 1.09 \times 10^7$). The energy is dominated by the mean-flow mode, so energy thresholds are reached with very few modes, while the error keeps decreasing quickly with r (about 37 at $r = 10$).
- Rank 10 gives a visually accurate reconstruction of every snapshot, compressing 89351×151 numbers to $10 \times (89351 + 151)$.